Практическое руководство по минимизации расхода токенов LLM в IDE

Введение: экономика токенов

Ключевой принцип: каждый токен стоит денег и времени. Оптимизация — это не просто экономия, это повышение эффективности разработки.

Базовая математика

- Claude Sonnet 4: ~$3/$15 за 1M токенов (input/output)
- Gemini 2.0 Flash: ~$0.075/$0.30 за 1M токенов
- Один типичный файл на 500 строк ≈ 2000-3000 токенов
- Одна IDE-сессия с автозагрузкой контекста: 10k-50k токенов

---

1. Промт-инжиниринг и управление контекстом

1.1 Архитектура эффективного промта

❌ Анти-паттерн (расточительный промт)

```
Привет! Я работаю над проектом на Python с использованием FastAPI.
У меня есть файл users.py, где реализована логика работы с пользователями.
Мне нужно добавить функцию для обновления email пользователя.
Можешь помочь? Вот весь файл:

[вставляется 500 строк кода]

Пожалуйста, напиши функцию update_user_email.
```

Расход: ~2500 токенов

✅ Оптимизированный промт

```
Task: Add update_user_email function
Context: users.py, FastAPI, SQLAlchemy ORM
Pattern: Similar to existing update_user_name at line 145
Requirements:
- Validate email format
- Check uniqueness
- Return updated user
Location: After line 145
```

Расход: ~50 токенов | **Экономия: 98%**

1.2 Системные промты и guardrails

Используйте .cursorrules, .clinerules или аналоги для установки глобальных правил:

```yaml
# .cursorrules
You are a code assistant. Rules:
1. Never repeat code back unless explicitly asked
2. Provide diffs, not full files
3. Reference line numbers: "After line X, add..."
4. Ask clarifying questions BEFORE generating
5. Use placeholders for obvious imports/boilerplate
6. Max response: 50 lines unless specified

Default output format: unified diff
Language: match project (detect from context)
```

Эффект: снижение output-токенов на 60-80%

1.3 Техники сокращения контекста

Метод 1: Локализация через навигацию

```
# Вместо вставки файла
"Open src/api/users.py, navigate to UserService class,
method create_user. Add email validation before line 78."
```

Метод 2: Сигнатуры вместо реализаций

```
Current structure:
```python
class UserService:
    def create_user(data: dict) -> User: ...
    def get_user(id: int) -> User | None: ...
```

Task: Add update_user method between create and get
```

Метод 3: Diff-only контекст

```
Current (line 45-50):
- return user
+ Proposed change: add logging before return
```

1.4 Повторное использование контекста

Стратегия "Context Anchoring"

```
# Первый промт - устанавливаем якорь
"Analyze src/models/user.py structure.
Remember it as [ANCHOR:USER_MODEL]"

# Последующие промты
"Using [ANCHOR:USER_MODEL], add is_active field"
"Using [ANCHOR:USER_MODEL], create migration"
```

Важно: Claude/GPT не хранят якоря между сессиями. Используйте:

- MCP для persistent memory
- Локальные embeddings (см. раздел 4)
- Project briefs в начале сессии

1.5 Минимизация истории диалога

Паттерн "Conversation Pruning"

```
# После каждых 5-7 сообщений
"Summarize our work in 3 bullet points for context reset"

# Новая ветка диалога
[Summary from previous] → minimal context
"Continuing: now implement feature X"
```

Cursor/IDEs: используйте

- "New Chat" вместо длинного треда
- Composer для изолированных задач
- Cmd+L для новых веток

---

2. Настройки IDE и LLM-клиентов

2.1 Cursor AI - критические параметры

Settings → Features

```json
{
  "cursor.chat.contextLength": "low",  // вместо "high"
  "cursor.chat.includeProjectStructure": false,
  "cursor.chat.autoIncludeFiles": false,  // КЛЮЧЕВОЕ
  "cursor.chat.maxFiles": 3,
  "cursor.cpp.disabledLanguages": ["markdown", "json", "log"]
}
```

Эффект: снижение контекста с 20-30k до 2-5k токенов

.cursorignore - агрессивная стратегия

```
# Базовые исключения
node_modules/
.git/
dist/
build/

# Дополнительные для снижения токенов
*.lock
*.log
*.md          # документация обычно не нужна в контексте
*.test.ts     # тесты отдельно
*.spec.ts
migrations/   # старые миграции
__pycache__/
.env*
```

2.2 Antigravity Dev - специфика

```yaml
# .antigravity/config.yml
context:
  mode: minimal
  auto_include: false
  max_files: 2

agent:
  execution_mode: instruction  # не full_generation
  diff_only: true

memory:
  use_embeddings: true
  max_history: 10
```

2.3 VS Code + Continue / Cody

```json
// settings.json
{
  "continue.contextLength": 4096,  // вместо default 8192
  "continue.useAutocompletion": false,  // дорого!
  "cody.experimental.foldingRanges": true,  // сворачивает неактивный код
  "cody.contextSelection": "manual"
}
```

2.4 Режимы работы: стоимость

Режим	Токены/запрос	Когда использовать
Chat (normal)	2k-10k	Вопросы, планирование
Composer	5k-20k	Многофайловые изменения
Agent mode	20k-100k	Комплексные задачи
Autocomplete	500-2k/comp	Редко, для бойлерплейта

Правило: начинайте с Chat, эскалируйте только при необходимости.

---

3. Работа с кодом и изменениями

3.1 Формулирование задач без полного кода

Анти-паттерн

```
Вот мой компонент React:
[вставка 200 строк JSX]
Добавь валидацию формы
```

Паттерн "Specification First"

```
File: src/components/UserForm.tsx
Current: form with name, email inputs (lines 15-40)
Task: Add client-side validation
Rules:
- name: required, 2-50 chars
- email: valid format
Implementation: use react-hook-form (already imported)
Output: Only validation schema + onChange handlers
```

3.2 Локализация изменений

Техника "Surgical Edits"

```
Target: src/api/auth.ts
Function: loginUser (line 23-45)
Change: Line 34
  FROM: if (!user) throw new Error()
  TO: if (!user) throw new UnauthorizedError()
Note: UnauthorizedError imported at line 3
```

Токены: 40 vs 2000 (полный файл)

Использование LSP-информации

```
# Cursor/IDEs предоставляют
"Go to definition of UserService.createUser"
"Show all references to EMAIL_REGEX"

# Используйте это вместо вставки кода
"Modify UserService.createUser to use the EMAIL_REGEX
constant (defined at constants.ts:12)"
```

3.3 Inline-инструкции как контекст

Паттерн "Code-as-Prompt"

```typescript
// src/services/user.ts

export class UserService {
  // LLM: Add method here - getUsersByRole(role: string)
  // Return: User[], filter by user.role, handle empty case

  async createUser(data: CreateUserDto) {
    // existing implementation
  }
}
```

Промт к LLM:

```
"Implement TODOs in src/services/user.ts"
```

Преимущества:

- Контекст уже в файле
- Не нужно объяснять структуру

- 90% экономия токенов

3.4 Refactor vs Generation

Задача	Refactor (дешевле)	Generation (дороже)
Переименование	✅ IDE Refactor	❌
Изменение сигнатуры	✅ IDE + LLM help	❌
Новая функция	⚠️ Скелет вручную	✅
Новый модуль	❌	✅

Правило: если IDE может сделать за секунду — не используйте LLM.

---

4. Инструменты, расширения и MCP

4.1 Плагины для снижения токенов

Cursor Extensions (рекомендуемые)

- **Context Pruner**: автоматически удаляет дубли
- **Smart Ignore**: ML-based .cursorignore
- **Token Counter**: real-time мониторинг

VS Code Extensions

- **LLM Context Optimizer** (гипотетический, концепт)
- **Codebase Digest**: создает сжатые описания
- **Embeddings Cache**: локальное кэширование

4.2 MCP (Model Context Protocol)

Архитектура с MCP для снижения токенов

```
┌─────────────────┐
│  IDE/Chat  │
└─────────────────┘
      │ Короткие промты
      ▼
┌─────────────────┐
│ MCP Server  │ ⟵ Хранит контекст, embeddings
└─────────────────┘
      │ Подает только релевантное
      ▼
┌─────────────────┐
│  LLM API  │
└─────────────────┘
```

Пример использования

```typescript
// MCP хранит "project knowledge"
const mcp = new MCPClient()

// Первый запрос - строим базу
await mcp.index("./src", {
  exclude: ["tests", "node_modules"],
  createEmbeddings: true
})

// Дальнейшие запросы - только релевантное
const context = await mcp.retrieve("user authentication", {
  maxFiles: 2,
  maxTokens: 3000
})

// В LLM уходит минимум
llm.chat(`${context}\n\nTask: add 2FA support`)
```

Экономия: 70-90% токенов для больших проектов

4.3 Memory и Embeddings

Local Embeddings Strategy

```python
```

```
# Предварительная подготовка (офлайн)
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')

# Индексируем кодовую базу
codebase_chunks = split_files_into_chunks("./src")
embeddings = model.encode(codebase_chunks)

# В runtime
def get_relevant_context(query, max_tokens=2000):
    query_emb = model.encode(query)
    similarities = cosine_similarity(query_emb, embeddings)
    top_chunks = get_top_k(similarities, k=3)
    return truncate_to_tokens(top_chunks, max_tokens)
```

Tools для этого

- **ChromaDB** + Cursor
- **Pinecone** для команд
- **LlamaIndex** для локальной индексации

4.4 Автоматизация правил экономии

Pre-commit Hook для проверки промтов

```bash
#!/bin/bash
# .git/hooks/pre-commit

# Проверяем, что в inline-промтах нет огромного кода
for file in $(git diff --cached --name-only); do
  grep -n "LLM:" "$file" | while read line; do
    context_size=$(echo "$line" | wc -c)
    if [ $context_size -gt 500 ]; then
      echo "Warning: Large LLM context in $file"
      echo "Consider splitting or using references"
    fi
  done
done
```

---

5. Организационные и процессные приёмы

5.1 Prompt Rules для команды

Создайте `PROMPT_GUIDELINES.md` в корне проекта:

```markdown
# LLM Usage Guidelines

## When to use LLM
✅ New features, complex logic, API design
❌ Simple renaming, formatting, adding imports

## Prompt Structure (mandatory)
1. File path + specific location
2. Task (one sentence)
3. Context (what exists)
4. Output format (diff/code/explanation)

## Token Budget per Task
- Simple change: <100 tokens context
- Medium task: <1000 tokens
- Complex: <5000 tokens (consider breaking down)

## Code Context Rules
- Never paste full files >50 lines
- Use line numbers and signatures
- Reference existing patterns: "like function X at line Y"

## Examples
Good: "Add error handling to api.ts:45, pattern from utils.ts:12"
Bad: [pastes 500 lines] "fix bugs"
```

5.2 Чеклист для разработчика

Перед отправкой промта:

- [ ] Могу ли я сократить контекст в 2 раза?
- [ ] Указал ли я точное местоположение (файл:строка)?
- [ ] Использую ли я ссылки вместо вставки кода?
- [ ] Нужен ли мне весь output или только diff?
- [ ] Пробовал ли я сначала IDE-рефакторинг?

После получения ответа:

- [ ] Записать токены в лог (для анализа)

- [ ] Если ответ избыточен — feedback модели
- [ ] Обновить .cursorrules на основе опыта

5.3 Code Review для LLM-использования

```yaml
# .github/workflows/llm-review.yml
name: LLM Cost Review

on: [pull_request]

jobs:
  analyze:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2

      - name: Check for embedded LLM prompts
        run: |
          # Поиск больших inline-промтов
          rg "LLM:|TODO:" --stats | analyze_size.py

      - name: Estimate token cost
        run: |
          # Оценка, сколько токенов ушло на генерацию
          python scripts/estimate_tokens.py

      - name: Comment on PR
        if: cost > threshold
        run: |
          gh pr comment ${{ github.event.pull_request.number }} \
            --body "⚠️ High LLM token usage detected"
```

5.4 Когда НЕ использовать LLM

Задача	Инструмент	Почему
Форматирование	Prettier/Black	Мгновенно, бесплатно
Переименование	IDE Refactor	Безопаснее, быстрее
Добавление import	IDE Auto-import	Встроено
Типовые CRUD	Codegen/snippets	Детерминировано
Исправление typo	Spell checker	Очевидно

Правило: если решение детерминированное и автоматизируемое — не тратьте токены.

---

6. Практические примеры и сравнения

6.1 Кейс 1: Добавление API endpoint

❌ **Неоптимизированный подход**

```
Промт: "Вот мой router.ts [3000 строк], добавь новый endpoint для получения статистики пользователя"

Расход:
- Input: 12,000 токенов (файл + промт)
- Output: 8,000 токенов (весь новый роутер)
Total: 20,000 токенов
Cost (Claude Sonnet 4): $0.06 + $0.12 = $0.18
```

✅ **Оптимизированный подход**

```
Промт:
"File: src/api/router.ts
Add after line 156 (userRoutes group):

router.get('/users/:id/stats', getUserStats)

Pattern: match existing routes (e.g., line 145)
Controller: create in controllers/stats.ts
Response: { views: number, posts: number, lastActive: Date }"

Расход:
- Input: 150 токенов
- Output: 500 токенов (только новый код)
Total: 650 токенов
Cost: $0.0005 + $0.0075 = $0.008
```

Экономия: 96% ($0.172 и 30x меньше токенов)

6.2 Кейс 2: Рефакторинг

❌ **Неоптимизированный**

```
"Вот компонент [500 строк]. Разбей на подкомпоненты"

Расход: ~25,000 токенов
Время: 30 сек
Качество: может пропустить зависимости
```

✅ **Оптимизированный (гибридный подход)**

```
Шаг 1 (IDE): Extract Component → создать Header.tsx вручную
Шаг 2 (LLM): "Complete Header.tsx based on lines 15-45 from App.tsx"
Шаг 3 (IDE): Update imports автоматически

Расход: ~2,000 токенов
Время: 15 сек
Качество: выше (IDE гарантирует связность)
```

Экономия: 92%

6.3 Кейс 3: Debugging

❌ **Неоптимизированный**

```
"У меня ошибка [вставка stacktrace + 5 файлов по 200 строк]"

Расход: 50,000+ токенов
Проблема: модель теряется в коде
```

✅ **Оптимизированный**

```
"Error: TypeError at user.service.ts:67
Line: const email = user.profile.email
Context: user.profile is null (comes from DB, line 45)
Question: Should we add null-check or fix DB query?

Relevant code:
L45: const user = await db.users.findOne(id)
L67: const email = user.profile.email  // crash here"

Расход: 200 токенов
```

Экономия: 99.6%

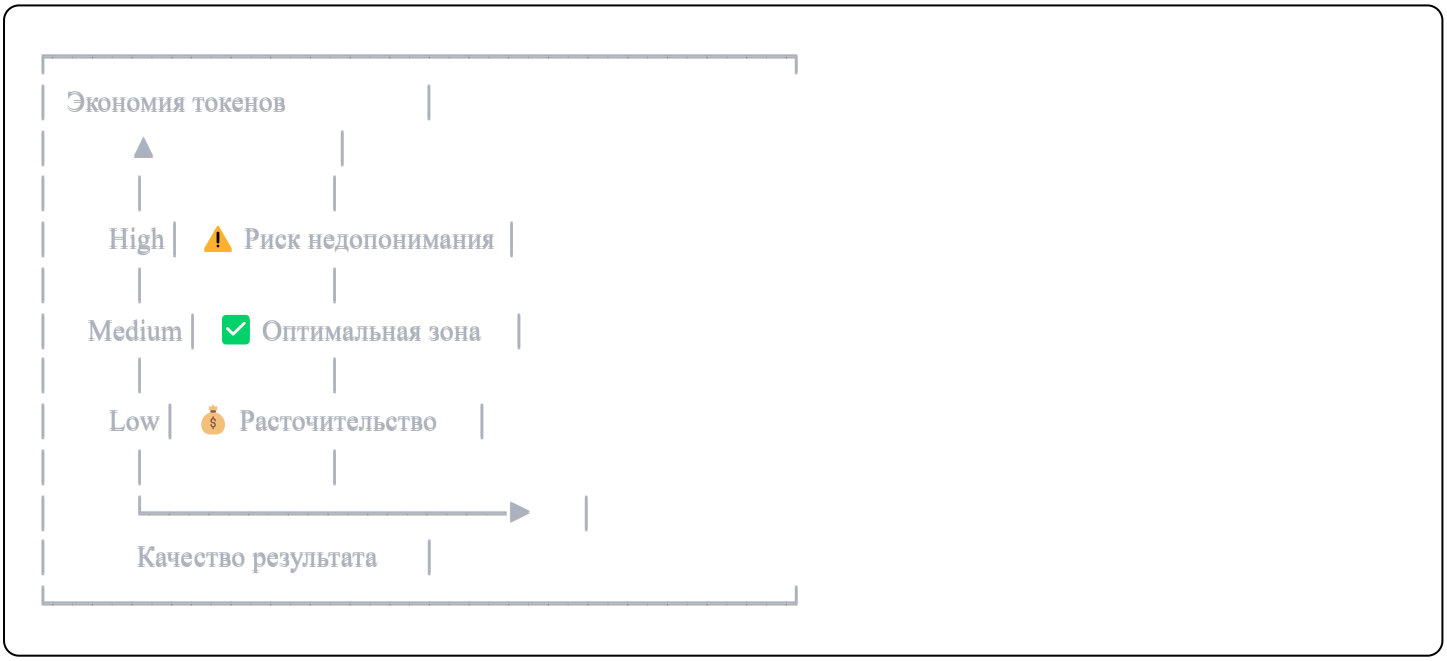
6.4 Сравнение: Claude vs Gemini vs GPT

Аспект	Claude Sonnet 4	Gemini 2.0 Flash	GPT-4o
Цена input	$3/1M	$0.075/1M	$5/1M
Цена output	$15/1M	$0.30/1M	$15/1M
Context window	200k	1M	128k
Оптимальный use-case	Качество кода	Быстрые итерации	Комплексный анализ
Стратегия экономии	Минимизировать output	Использовать широкий контекст	Короткие сессии

Рекомендация по модели:

- **Прототипирование**: Gemini Flash (дешево, быстро)
- **Production код**: Claude Sonnet (качество)
- **Code review**: GPT-4o (глубокий анализ)

6.5 Trade-offs: Экономия vs Качество



Оптимальная зона: 80% экономии при 95% качества

Где НЕ жалеть токены

1. **Критический код** (auth, payments)
2. **Новая архитектура** (нужен полный контекст)
3. **Debugging продакшена** (цена ошибки > цена токенов)

4. **Обучение** (лучше переплатить, чем не понять)

Где экономить агрессивно

1. **Рутинные задачи** (CRUD, boilerplate)
2. **Эксперименты** (может не пригодиться)
3. **Рефакторинг** (IDE справится лучше)
4. **Документация** (генерировать офлайн)

---

7. Метрики и мониторинг

7.1 Что отслеживать

```typescript
// Пример логирования
interface LLMUsageLog {
  timestamp: Date
  task: string
  inputTokens: number
  outputTokens: number
  cost: number
  model: string
  satisfied: boolean  // было ли решение удачным
}

// Анализировать еженедельно
function analyzeUsage(logs: LLMUsageLog[]) {
  return {
    totalCost: sum(logs.map(l => l.cost)),
    avgTokensPerTask: avg(logs.map(l => l.inputTokens + l.outputTokens)),
    costPerSuccessfulTask: totalCost / logs.filter(l => l.satisfied).length,
    topExpensiveTasks: logs.sort((a,b) => b.cost - a.cost).slice(0, 10)
  }
}
```

7.2 Целевые показатели

Метрика	Начинающий	Опытный	Expert
Токенов/запрос	5000-10000	1000-3000	<1000
Стоимость/день	$5-10	$2-4	<$1

Метрика	Начинающий	Опытный	Expert
Решенных задач/$	5-10	20-30	50+

7.3 Dashboard для команды

```python
# dashboard.py - простой мониторинг
import streamlit as st
import pandas as pd

logs = pd.read_csv("llm_usage.csv")

st.title("LLM Usage Dashboard")

col1, col2, col3 = st.columns(3)
col1.metric("Total Cost", f"$ {logs['cost'].sum():.2f}")
col2.metric("Avg Tokens", f"{logs['tokens'].mean():.0f}")
col3.metric("Tasks", len(logs))

st.line_chart(logs.groupby('date')['cost'].sum())

# Топ дорогих запросов
st.subheader("Most Expensive Queries")
st.dataframe(logs.nlargest(10, 'cost')[['task', 'cost', 'tokens']])
```

---

8. Быстрая справка (Cheat Sheet)

Перед каждым запросом спроси себя

```
1. Контекст
  [ ] Минимально необходимый?
  [ ] Можно заменить ссылками?
  [ ] Нет дублей?

2. Формулировка
  [ ] Конкретная локация?
  [ ] Формат output указан?
  [ ] Примеры/паттерны указаны?

3. Альтернативы
  [ ] Может IDE справится?
```

```
  [ ] Можетググлить быстрее?
  [ ] Нужен ли вообще LLM?
```

Шаблоны промтов (копипаста)

```
# Добавление функции
File: <path>:<line>
Add: <function_name>
Signature: <signature>
Logic: <one line>
Pattern: like <existing_function> at line <N>

# Исправление бага
Error: <error message>
File: <path>:<line>
Current: <current code>
Fix: <what to do>

# Рефакторинг
Target: <path>:<lines>
Goal: <what to achieve>
Constraints: <limits>
Output: diff only

# Ревью кода
Diff: <git diff> (max 50 lines)
Focus: <security/performance/style>
Format: line numbers + suggestions
```

---

9. Продвинутые техники

9.1 Prompt Compression через модель

```python
```

```
# Используйте легкую модель для сжатия контекста
def compress_context(large_context: str) -> str:
    """Gemini Flash сжимает контекст для Claude"""
    prompt = f"""
    Summarize this code context in <500 tokens, preserve:
    - Key structures/classes
    - Important functions (name + signature only)
    - Dependencies
    - Location info (file:line)

    Context:
    {large_context}
    """

    compressed = gemini_flash.generate(prompt, max_tokens=500)
    return compressed

# Затем в дорогую модель
result = claude_sonnet.generate(f"{compress_context(code)}\n\n{task}")
```

Экономия: 70-80% на input-токенах дорогой модели

9.2 Streaming с ранним прерыванием

```typescript
// Не ждите полного ответа, если первые строки показали ошибку
const stream = await llm.streamChat(prompt)

let collectedTokens = 0
for await (const chunk of stream) {
  collectedTokens += chunk.tokens

  // Ранний выход если модель не поняла
  if (collectedTokens > 100 && chunk.text.includes("I don't have enough context")) {
    stream.abort()
    return refinePromptAndRetry()
  }

  // Или если уже получили нужное
  if (chunk.text.includes("```\n// Solution complete")) {
    stream.abort()
    break
  }
}
```

9.3 Batch Processing

```typescript
// Вместо 10 запросов по 1000 токенов
for (const file of files) {
  await llm.chat(`Refactor ${file}`)  // 10,000 токенов total
}

// Один запрос с batch
await llm.chat(`
Refactor these files (output: filename + changes only):
${files.map(f => `${f.path} (line ${f.targetLine})`).join('\n')}

Pattern: ${sharedPattern}
`)  // 3,000 токенов total
```

9.4 Кэширование на уровне промтов

```typescript
// Anthropic's Prompt Caching (beta)
const systemPrompt = `
You are a code assistant for this project:
${projectStructure}  // 10,000 токенов

${codeStandards}     // 5,000 токенов
`

// Первый запрос: платим полностью
await claude.chat(systemPrompt, "Task 1")  // 15,000 input токенов

// Следующие запросы: system prompt закэширован
await claude.chat(systemPrompt, "Task 2")  // ~100 input токенов
await claude.chat(systemPrompt, "Task 3")  // ~100 input токенов

// Экономия: 99% на повторяющемся контексте
```

---

10. Итоговый чеклист внедрения

Неделя 1: Базовая оптимизация

- [ ] Настроить .cursorignore / .clinerules
- [ ] Создать .cursorrules с правилами

- [ ] Отключить auto-include files
- [ ] Установить context length: low
- [ ] Обучить команду базовым паттернам

Ожидаемая экономия: 40-50%

Неделя 2: Процессы

- [ ] Создать PROMPT_GUIDELINES.md
- [ ] Внедрить чеклист перед запросом
- [ ] Настроить логирование токенов
- [ ] Провести code review LLM-использования

Ожидаемая экономия: 60-70%

Месяц 1: Инструменты

- [ ] Внедрить MCP для больших проектов
- [ ] Настроить локальные embeddings
- [ ] Автоматизировать анализ usage
- [ ] Создать dashboard мониторинга

Ожидаемая экономия: 75-85%

Квартал 1: Культура

- [ ] Регулярные ретро по LLM-usage
- [ ] База знаний с best practices
- [ ] Gamification экономии токенов
- [ ] Sharing успешных паттернов

Ожидаемая экономия: 85-90%

Заключение

Ключевые принципы

1. **Контекст — враг №1**: каждый лишний токен стоит денег
2. **Локализация > генерация**: точечные изменения дешевле полной генерации
3. **IDE first, LLM second**: используйте встроенные инструменты
4. **Измеряйте всегда**: нельзя оптимизировать то, что не измеряется
5. **Автоматизируйте**: правила экономии должны применяться автоматически

ROI оптимизации

Типичная команда из 5 разработчиков:

- Без оптимизации: $500-1000/месяц
- С базовой оптимизацией: $200-400/месяц
- С продвинутой оптимизацией: $50-150/месяц

Время на внедрение: 2-4 недели
Окупаемость: 1-2 месяца

Дальнейшие шаги

1. Начните с самого простого: .cursorrules + .cursorignore
2. Измеряйте baseline (текущие расходы)
3. Внедряйте улучшения постепенно
4. Делитесь результатами с командой
5. Итерируйте на основе метрик

Помните: экономия токенов — это не просто про деньги, это про скорость, фокус и эффективность разработки.

Документ актуален на январь 2025. Следите за обновлениями моделей и инструментов.